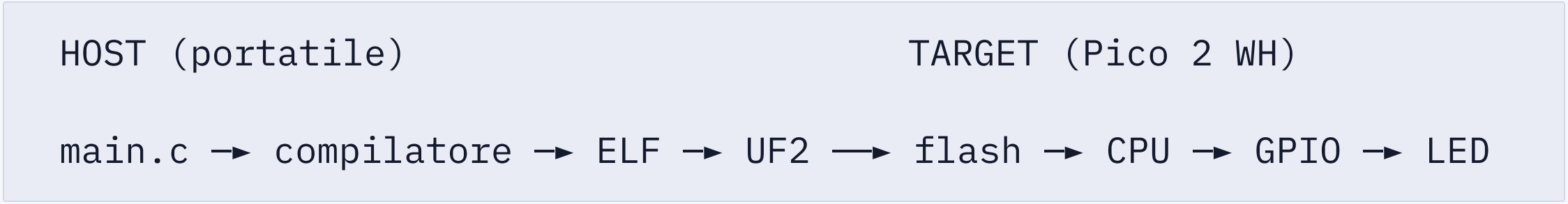
# Dal codice sorgente a un sistema embedded in esecuzione

Embedded and IoT Intelligent Systems Programming · Lab 1 · 16 ottobre 2026



# Un programma, seguito fino in fondo




Seguire un programma dal codice sorgente a un comportamento osservabile su hardware reale.

# Il percorso di questo lab

```
BUILD → ISPEZIONA → FLASH → OSSERVA → MODIFICA → MISURA
```

| FASE | TRACCIA |
|---|---|
| Preparare l'ambiente | Parte 0 |
| Build, ispezione | Parti 1 e 2 |
| Flash, osservazione | Parte 3 |
| Modifica | Parte 4 |
| Misura | Parte 5 |

# Compilazione

Un processore non capisce il C. Esegue il **codice macchina**, una sequenza di numeri che codificano operazioni molto semplici, come sommare due valori o leggere un indirizzo di memoria. Un **compilatore** traduce i file sorgente C in codice macchina, e un **linker** unisce i pezzi tradotti in un unico programma.

Questo è già stato fatto nei corsi precedenti: lanciare `gcc` su un portatile e poi eseguire il risultato è esattamente questo processo.

```
main.c ──► compilatore ──► codice macchina
```

# Cross-compilazione

Il codice macchina appartiene a una sola famiglia di processori. Un programma compilato per il processore x86-64 di un portatile non può girare su un Arm Cortex-M33, perché i due capiscono istruzioni diverse.

Un **cross-compilatore** gira su una macchina, l'**host**, ma produce codice per un'altra macchina, il **target**. In questo lab l'host è il computer usato per compilare il programma, e il target è il chip sulla Pico.

```
host ──► cross-compilatore ──► codice macchina per il target
```

# Microcontrollore e scheda

L'RP2350 è un **microcontrollore**: un singolo chip che contiene il processore, qualche centinaio di kilobyte di RAM e i circuiti che comunicano con il mondo esterno. La memoria che conserva il programma è un chip flash separato, collegato al microcontrollore sulla scheda.

La **Raspberry Pi Pico 2 WH** è la scheda costruita attorno all'RP2350. Aggiunge il connettore USB, la regolazione dell'alimentazione, il chip flash e i pin lungo i bordi. La H indica che i connettori dei pin sono già saldati, e la W indica che la scheda ha anche un chip wireless, che in questo lab non viene usato.

# Firmware

Il **firmware** è il programma che vive dentro un dispositivo embedded. Non c'è un disco, non c'è un login e non c'è un programma che avvia gli altri: il codice è conservato nella memoria flash, e il processore inizia a eseguirlo non appena arriva l'alimentazione.

In questo corso il firmware è costruito su **Zephyr**, un sistema operativo real-time che fornisce un kernel, driver e librerie. Il codice dell'applicazione, `main.c`, viene collegato alle parti di Zephyr di cui ha bisogno, e il risultato è un'unica immagine che viene scritta sulla scheda.

# Memoria flash e flashing

La **memoria flash** conserva il proprio contenuto quando manca l'alimentazione, a differenza della RAM, che senza alimentazione non mantiene i dati. Il programma deve quindi essere conservato nella flash.

Il **flashing** significa scrivere l'immagine in quella memoria. Sulla Pico la scrittura è fatta dalla **boot ROM**, un piccolo programma scritto nel chip in fabbrica, che può presentare la scheda al computer come un'unità USB. Una volta scritta l'immagine, la scheda la esegue a ogni accensione, finché una nuova immagine non la sostituisce.

# GPIO

**GPIO** sta per general-purpose input/output. Un GPIO è un pin il cui ruolo è deciso dal software. Configurato come uscita, il programma lo porta a una tensione alta (3,3 V) o a una tensione bassa (0 V). Configurato come ingresso, il programma legge quale delle due tensioni è presente.

Collegare un LED, attraverso una resistenza, a un pin di uscita permette al programma di accendere e spegnere il LED. È il risultato osservabile di tutto il lab: una riga di codice C che cambia una tensione su un pin.

# Due numeri per ogni pin

Ogni pin GPIO ha due numeri: il **pin fisico**, la sua posizione lungo il bordo della scheda (da 1 a 40), e il **nome GPIO**, come GP15, che usa il software. I pin di alimentazione e di massa hanno una posizione ma non un nome GPIO.

I due numeri non sono collegati. Il LED è pilotato da GP15, che è il pin fisico 20, mentre il pin fisico 15 è GP11. Confonderli è un errore di cablaggio facile da fare.

```
pin fisico 20  =  GP15   (LED, attraverso la resistenza)
pin fisico 18  =  GND
```

# Console seriale e logging

Una scheda embedded non ha uno schermo. Per vedere cosa fa il programma, questo invia del testo su una linea di comunicazione, e il computer riceve quel testo con un programma terminale.

Sulla Pico questa linea è una **porta seriale virtuale** che passa dallo stesso cavo USB usato per il flashing. Il **logging** consiste nello stampare dal codice brevi messaggi con data e ora, nei punti che contano. È spesso il primo modo, e a volte l'unico, per guardare dentro un programma in esecuzione.

# Host e target a confronto

| | HOST | TARGET |
|---|---|---|
| Macchina | portatile | Raspberry Pi Pico 2 WH |
| Processore | CPU x86-64 o Arm del portatile | RP2350, Arm Cortex-M33 |
| Esegue | editor, compilatore, terminale | firmware: applicazione + Zephyr RTOS |
| Memoria | gigabyte | 520 KB di RAM, 4 MB di flash |

Il compilatore gira sull'host (qui, dentro il container di build).
Il firmware gira sul target. La Pico non compila niente.

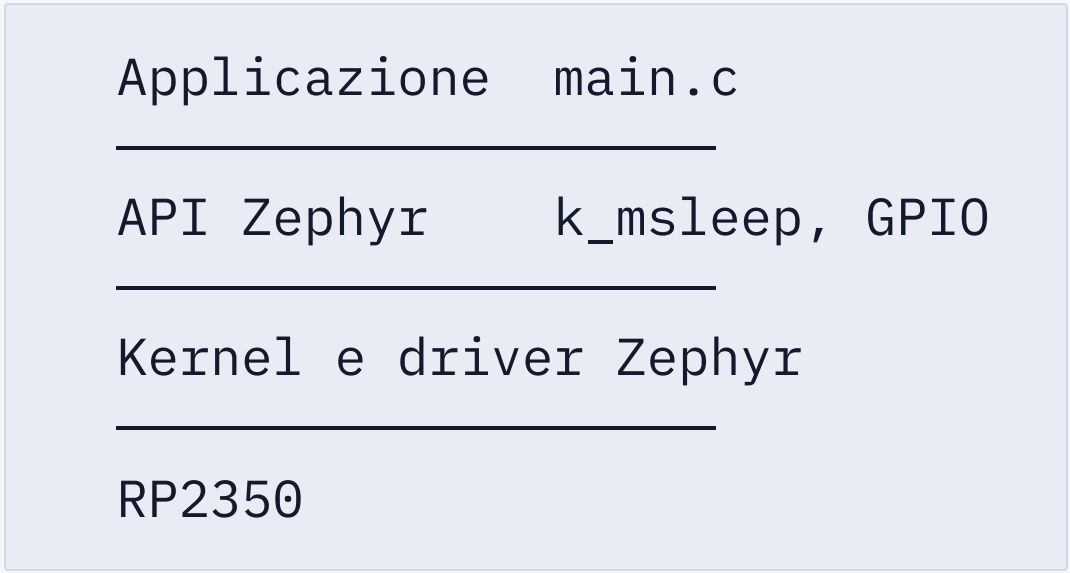
DA FARE ORA

# Quale compilatore è questo?

```
./bin/course uname -m
./bin/course arm-zephyr-eabi-gcc -dumpmachine
```

- 1° comando: dove gira il compilatore (il container)
- 2° comando: per cosa viene compilato il codice

# Lo stack software




Strumenti di build

- `west`
- CMake
- Ninja
- GCC (cross)

Girano tutti nel container del corso.

DeviceTree e Kconfig: Lab 3.

# Un'applicazione Zephyr minima

```
app/
├── CMakeLists.txt      cosa compilare
├── prj.conf            quali funzioni di Zephyr includere
├── boards/             su quale pin è il LED
└── src/
    └── main.c          il programma
```

Un file, un ruolo. Non serve altro per produrre un firmware.

# main.c

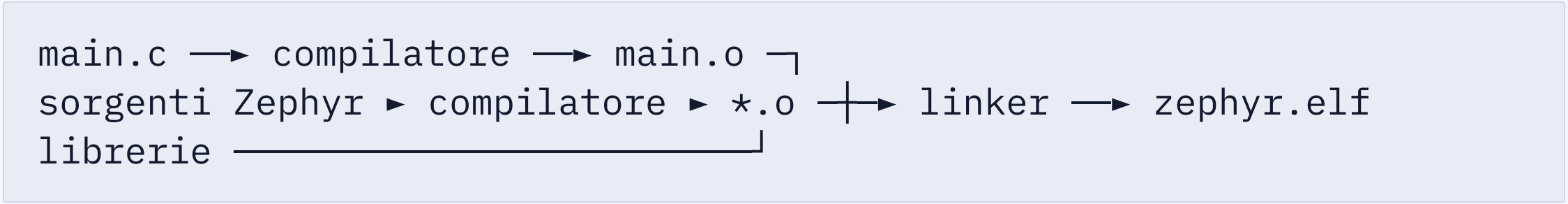
```
for (uint32_t n = 0;; n++) {
    uint32_t start = k_cycle_get_32();

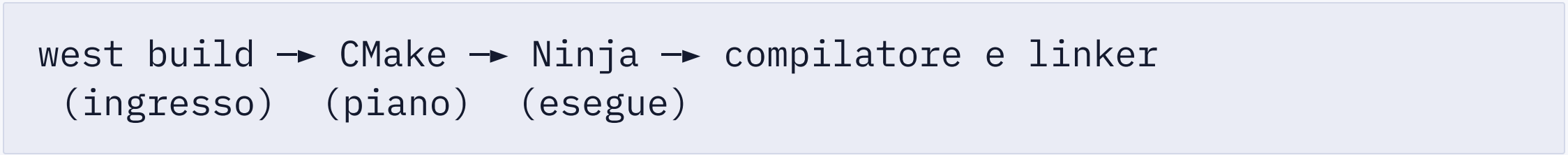
    gpio_pin_toggle_dt(&led);
    k_msleep(BLINK_PERIOD_MS);

    uint32_t us = k_cyc_to_us_near32(k_cycle_get_32() - start);
    LOG_INF("toggle %u: asked %d ms, measured %u us", n, BLINK_PERIOD_MS, us);
}
```

C ordinario, compilato contro Zephyr. `led` viene dall'alias devicetree `led0`.

# Cosa succede durante la build




CMake pianifica la build una sola volta. Ninja esegue solo ciò che è cambiato.

# L'ELF contiene più di ciò che si scrive in flash

`zephyr.elf` è l'eseguibile della toolchain: codice macchina, simboli, sezioni e informazioni di debug. La boot ROM accetta un'immagine flash, non un ELF.

```
linker ──┬──► zephyr.elf ──┬──► zephyr.bin   byte grezzi
         │                 ├──► zephyr.hex   byte come testo, con indirizzi
         │                 └──► zephyr.uf2   formato della boot ROM della Pico
         └──► zephyr.map   rapporto: dove il linker ha messo ogni cosa
```

# Leggere l'output di size

Lo strumento `size` divide il programma in tre parti. `text` è il codice e i dati in sola lettura, conservati nella flash. `data` è l'insieme delle variabili che partono con un valore diverso da zero: i valori iniziali sono conservati nella flash e copiati nella RAM all'avvio. `bss` è l'insieme delle variabili che partono da zero, e occupano spazio solo nella RAM.

La flash usata è quindi circa `text + data`, e la RAM usata è circa `data + bss`. L'ultima colonna, `dec`, è solo la somma delle tre e non descrive nessuna delle due memorie.

DA FARE ORA

# Ispezionare prima di scrivere in flash

```
$ arm-zephyr-eabi-readelf -h zephyr.elf | grep -E 'Class|Machine'
  Class:                             ELF32
  Machine:                           ARM
$ arm-zephyr-eabi-size zephyr.elf
   text    data     bss     dec
  55924     372   14430   70726
```

Quale architettura? Flash usata = `text` + `data`: quanto dei 4 MB?

DA FARE ORA

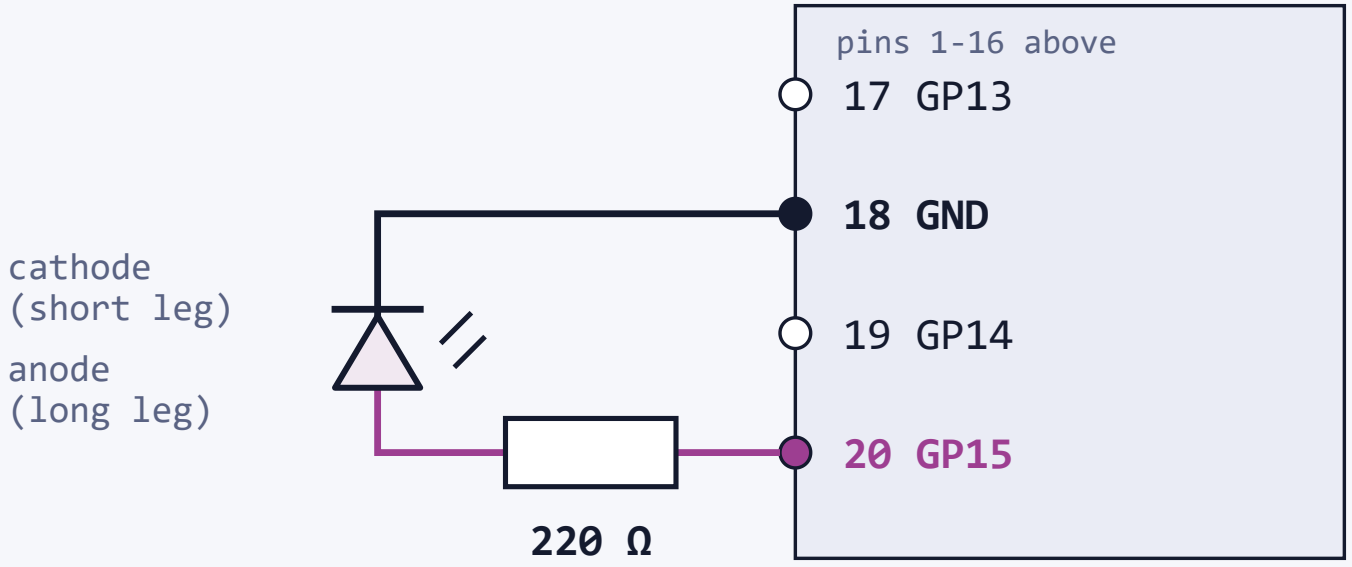
# Elenco dei materiali

| ELEMENTO | QTÀ | NOTA |
|---|---|---|
| Raspberry Pi Pico 2 WH | 1 | connettori saldati |
| Cavo micro-USB per dati | 1 | i cavi solo ricarica non vanno |
| Breadboard | 1 | la Pico si inserisce sopra |
| LED, qualsiasi colore | 1 | gamba lunga = anodo |
| Resistenza da 220 Ω | 1 | limita la corrente del LED |

DA FARE ORA

# Cablaggio

**GP15 è il pin fisico 20.** Il pin fisico 15 è GP11.



DA FARE ORA

# Dal file all'hardware

1. Tenere premuto BOOTSEL, collegare il cavo USB.
2. La boot ROM parte e la Pico appare come unità USB `RP2350`.
3. Copiare `zephyr.uf2` sull'unità.
4. La boot ROM lo scrive in flash e riavvia il chip.
5. Il programma parte.

Una scheda vuota mostra l'unità senza il pulsante. Dopo il primo programma, tenere premuto BOOTSEL a ogni collegamento.

DA FARE ORA

# Leggere la porta seriale

La scheda appare sul computer come una porta seriale: un nome di dispositivo che un programma terminale può aprire e leggere. Il programma sulla scheda vi scrive il log, e il terminale stampa ciò che arriva.

```
Linux     /dev/ttyACM0          picocom -b 115200 /dev/ttyACM0
macOS     /dev/cu.usbmodem*     screen /dev/cu.usbmodem* 115200
Windows   COMx                  PuTTY o Tera Term, Serial, 115200
```

115200 è un'impostazione che i terminali chiedono. Aprire la porta fa partire anche il log.

# Osservare un programma in esecuzione

## Fisico

Il LED su GP15.

```
[00:00:03.014,000] <inf> lab01:
  toggle 0: asked 1000 ms,
  measured 1000045 us
```

## Software

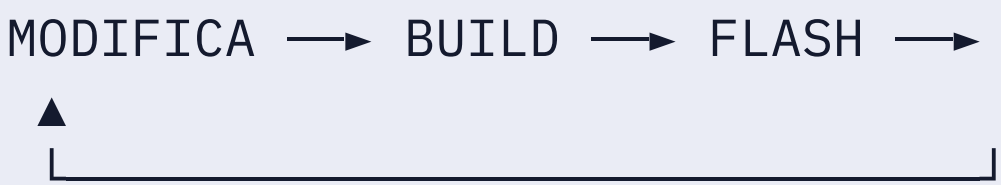
Seriale USB e logging.

Il software embedded ha bisogno di osservabilità. Senza questi due canali, la scheda è una scatola chiusa.

DA FARE ORA

# Cambiare una riga

```
#define BLINK_PERIOD_MS 1000   →   #define BLINK_PERIOD_MS 250
```




Sorgente, binario, flash e comportamento cambiano.
Il ciclo si ripete in ogni lab.

DA FARE ORA

# Tempo richiesto e tempo reale

```
k_msleep(1000);

asked:    1000 ms
measured: 1000043 us
```

L'intervallo inizia prima del toggle e finisce dopo il ritorno dallo sleep, quindi comprende entrambi.

Raccogliere 20 righe. Annotare l'errore minimo e massimo e un motivo.

DA FARE ORA

# Misurare invece di supporre

Quanto dura una chiamata a `gpio_pin_toggle_dt()`?

```
una operazione      troppo breve per una lettura singola affidabile
1000 operazioni     misurabile
totale / 1000       costo di una
```

Racchiudere il ciclo con `k_cycle_get_32()` e `k_cyc_to_ns_near32()` (`k_cyc_to_us_near32()` restituisce microsecondi).

# Il percorso completo

```
main.c
  ▼
west → CMake → Ninja
  ▼
arm-zephyr-eabi-gcc
  ▼
zephyr.elf → zephyr.uf2
  ▼
boot ROM dell'RP2350 → flash
  ▼
la CPU esegue → GPIO → LED
```

# Cosa ha stabilito questo lab

- L'host non è il target; un cross-compilatore li collega.
- Una build trasforma il sorgente in codice per il target, e gli artefatti si possono ispezionare.
- Il firmware diventa comportamento osservabile attraverso il LED e il log.
- Lo sviluppo embedded è un ciclo di modifica, build, flash e osservazione.

**Lab 2:** più attività condividono un processore, ognuna con il proprio vincolo di tempo.